

of the `Club` class. For example, a client could maliciously or accidentally change the maximum number of members:

```
oProgrammingClub.maxMembers = 300
```

Further, suppose you know that the `Club` class represents the members as a list, and you know the name of the instance variable that represents the members. In that case you can write client code to add to the list of members directly, without making the method call, like so:

```
oProgrammingClub.memberList.append('Iwanna Join')
```

This line would push the membership over the intended limit because it avoids the code that ensures that the request to add the member is valid.

Client code using direct access could even cause an error inside the `Club` object. For example, the instance variable `self.maxMembers` is intended to be an integer. Using direct access, client code could change its value to a string. Any subsequent call to `addMember()` would crash at the first line of that method, where it attempts to compare the length of the list of members against the maximum number of members, because Python cannot compare an integer to a string.

Allowing direct access to instance variables from outside an object can be dangerous, bypassing safeguards that were designed to protect the data of an object.

Strict Interpretation with Getters and Setters

The strict approach to encapsulation says that client code *never* accesses an instance variable directly. If a class wants to allow client software to access the information held inside an object, the standard approach is to include a *getter* and a *setter* method in the class.

getter

A method that retrieves data from an object instantiated from a class.
